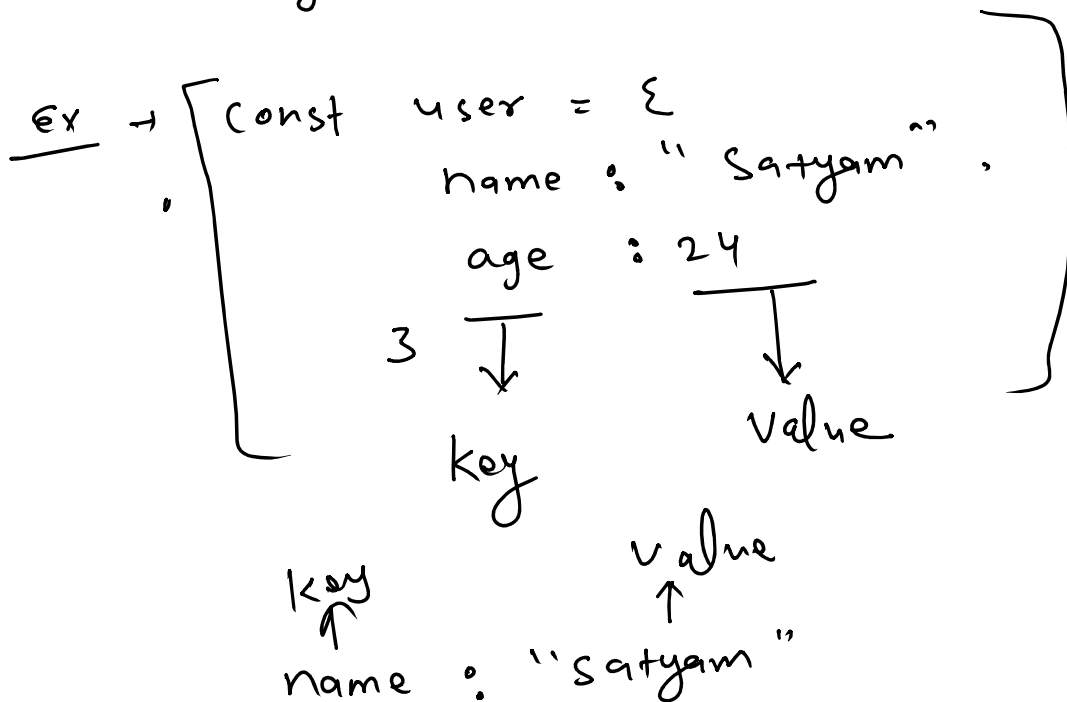


Objects, JSON, Spread, Rest & Destructuring ✓

Objects -

→ An object is a collection of key-value pairs used to store related data.

Key - value -



Creating an object

① objects literal

```
const user = {
  name: "Rahul",
  age: 23
}
```

⑦ Empty object

```
const student = {}
```

⑧ Using new object -

```
const user = new Object();
```

~~user~~

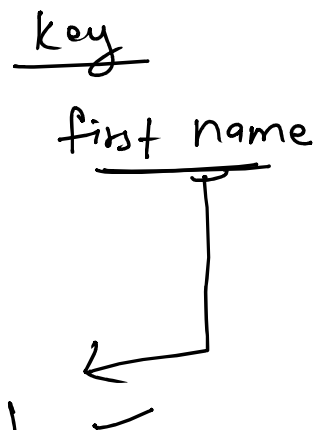
Accessing object properties

① Dot Notation -

```
console.log(user.name)
```

② Bracket Notation

```
console.log(user["name"])
```



Adding → user.role = "Developer"
↓
if exists then update otherwise
add

delete property

delete user.name ✓
↪ name key will be deleted

nested object →

→ An objects can contain another objects.

```
const user = {  
  name: "Suryam",  
  address: {  
    city: "Patna",  
    state: "Bihar"  
  }  
}
```

↪ user.address.city ↪
user.address.state

user.address.state

Object methods -

→ A function stored inside an object is called a method.

Dynamic key

→ const dkey = "name"
const user = {
 [dkey] : "Satyam"
}
✓ user.name ✓

in operator (True/false)

→ checking if a property/key exists.

("key" in objName)

object.keys() , object.values
↓ ↓

return an array
containing the object
keys → values

object.entries()

↳ Returns an array of key-value
pairs

Looping through an object

→ for in ✓

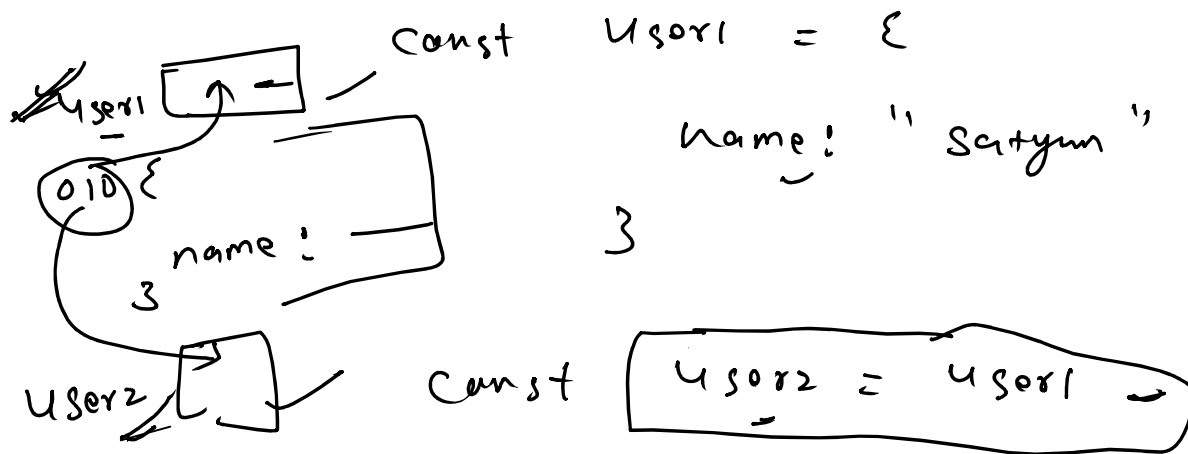
for (const kuchhbi in objname) {
 console.log(key, objname[*key*])
}

- # optinal chaining (?) ✓

→ &
optinal chaining (?) safely accesses
properties that may not exists.

console.log(user_address?.city)

#1 Object references -



#2 Object copy

Two types

one layer copy

① Shallow copy → ~~maximum~~

② Deep copy → fully copy.

#1 Shallow copy

→ A shallow copy creates a new outer object, but hosted objects are still shared.

⇒ Object spread (...) -

```

✓ const obj1 = {
  name: "Rahul",
  address: {
    city: "Noida"
  }
}

```

```

const obj2 = { ...obj1 }

```

```

obj2.address.city = "Putha"
    ↓
  changed obj1.

```

Deep copy

→ A deep copy creates a new object including nested object

① • Structured Clone (object-name) → Deep

② ⇒ JSON method →

```

✓ JSON.parse (JSON.stringify(user))

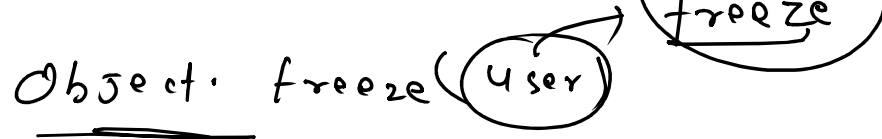
```

Object.freeze()

Object.freeze() =

↳ you can not add, update, delete properties

Object.freeze(user)



Object.seal()

↳ no add or, remove allowed

But can change value

↳ Object.seal(user)

JSON -

→ Stands for JavaScript Object Notation

→ It is a text-based format for storing and exchanging structured data

Ex

{
 "name" : "Satyam",
 "age" : 23,

}

↳ mostly used

```

    {
      "age": 23,
      "role": "Developer"
    }
  
```

used with APIs

± JSON.stringify()

Convert JS values to
JSON string

± JSON.parse()

↓
JSON string to
JS value

± Spread operator (...)

यह हमें मिलेगा

→ The spread operator is used to
expand or unpack values

spread = Expand (unpack)

→ mostly used with object and
Arrays

± Spread with Array

const arr1 = [1, 2, 3] ✓

const arr2 = [4, 5, 6] ✓

const combined = [...arr1, ...arr2]



[1, 2, 3, 4, 5, 6]

Spread with object

const obj1 = {a: 1, b: 2} ✓
const obj2 = {b: 3, d: 4} ✓

const combined = {...obj1, ...obj2}



{
 3 ✓

Rest operator/parameter → (...)

→ Rest parameter collects remaining arguments into an array -

→ Rest in function



→ Rest in function

function sum(...nums) {
 return nums.reduce((acc, el) => acc + el, 0)
}

array → (reduce)
↑
↓

Consider: log(sum(1, 2, 3, 4))
↓
10

→ Rest parameter must be last

Destructuring

→ Destructuring is a js feature. used to extract values from array or object

object Destructuring

→ const user = {
 name: "Satyan",
 ...}

```
name: "Satyan",  
age: 23,  
role: "Developer"  
}
```

```
const { name, age, role } = user  
↓  
log(name) → Satyan
```

```
const { name: userName } = user
```

```
const { name, city: "Patna" } = user  
↓ ↓  
default value
```

Array Destructuring -

```
const numbers = [10, 20, 30]
```

```
const [a, b, c] = numbers
```

→ skip

```
const [first,           , third] = numbers  
          ↓  
          skip
```

SKIP